

Artem Churilkin¹, Co-author Two¹, Co-author Three¹

THE VERIFIER BOTTLENECK: CAN AN AI TEACH ITSELF MATHEMATICS?

ABSTRACT. This work studies whether low-rank fine-tuning on exact, automatically verified composition trajectories can change the probability distribution of a small language model so that correct programs take higher positions on new tasks under an unchanged evaluation procedure. The experiments use Qwen3-0.6B [15] in a controlled environment of polynomial transformations over finite fields, where a solution is a sequence of three atomic operations chosen from five. For every task all 125 distinct programs of depth three were enumerated and scored exactly, so the main measure Hit@32 contains no sampler and no repeated candidates. Two branches of equal training budget were compared, an atomic control and fine-tuning on verified trajectories. Mean Hit@32 rose from 45.32 to 72.72 percent, a gain of 27.40 percentage points with a 95 percent t interval of [25.40, 29.40] over six independent runs and $p = 3.45 \times 10^{-7}$, positive in all six. A second series measured what the surrounding pipeline contributes: structured exploration carries no advantage over independent sampling at matched candidate diversity, and four hundred steps of training with a verifiable reward change held-out coverage by +0.22 points. Transfer to previously held local motifs was not confirmed in either series, and part of the atomic skills degraded, so the positive effect should be read as reliable consolidation of familiar families of compositions and transfer to new tasks, and not as the universal emergence of an arbitrary new compositional ability.

§1. INTRODUCTION

Modern language models can improve on formalisable tasks through cycles of generation, automatic checking and further fine-tuning: the model forms several candidates, a checking program picks out the correct ones, and those become new training material. Schemes of this kind underlie self-taught reasoning, self-consistent inference, step-level verification and training with a verifiable reward [5, 10, 14, 18].

The main question is what exactly changes after such training. In the first of two possible effects the model only raises the probability of solutions already

Key words and phrases: language model, skill composition, checking program, low-rank fine-tuning, exact ranking, finite field, Hit@32, reproducible experiment.

available to it but rare; in the second it begins to reproduce compositions of actions on new tasks whose input states were absent from the training set. Distinguishing these regimes matters for the theory of self-improvement and for the design of practical systems of mathematical inference [11, 17].

The project considers two control axes: the informativeness of the checking program, and the availability of diverse candidates together with the possibility of combining learned skills. Early experiments showed that an external widening of the search is easily confused with a real change in the parameters. The evaluation procedure is therefore fixed and never changed between the compared branches: for every task the same complete set of programs is built, and the model influences only their probability order. The aim is to test whether low-rank fine-tuning on exact, automatically verified composition trajectories raises the rank of correct programs on new tasks against additional atomic training of the same budget. A positive result raises a second question that experiment cannot answer, namely how much of the effect belongs to the fine-tuning and how much to the surrounding pipeline of search and reward, which Section 5.4 measures.

The contributions are four measurements. Fine-tuning on verified composition trajectories raises exact ranking of correct depth-3 programs by 27.40 percentage points against an atomic control of equal budget, 95 percent t interval [25.40, 29.40], positive in all six independent runs, carrying to new finite fields at 26.13 points. The advantage of structured exploration over independent sampling is accounted for by the number of distinct programs the search actually tries: at matched diversity it is -1.11 points, interval $[-4.22, +1.89]$. Four hundred steps of policy-gradient training with an exact binary reward leave held-out coverage unchanged within three percentage points, at $+0.22$, while at the arm where the preferences of the policy dominate the same runs make it 2.37 times more accurate per candidate and 2.10 times narrower. Supervision does not transfer to adjacent operation pairs withheld from it, at -16.02 and -11.80 points here and at -13.47 and -15.07 in the second series, where a control shows that adapters whose data contained those pairs reach Hit@32 of 0.935 and 0.933 on exactly the families that the withheld adapters fail.

§2. RELATED WORK

Self-training with automatic checking has four steps: the model forms candidates, an external algorithm checks each of them, the correct ones are stored as positive trajectories, and the parameters are updated so that their probability grows. Zelikman et al. [18] bootstrap reasoning traces this way, Wang et al. [14] aggregate over sampled traces, Lightman et al. [10] supervise the individual steps,

and Gulcehre et al. [5] formalise the loop as repeated growth and improvement stages. The reliability of the checking program is described by α , the probability of accepting a correct answer, and β , the probability of accepting an incorrect one: at $\alpha > \beta$ the signal carries information about correctness, at $\alpha = \beta$ it is uninformative, and at $\alpha < \beta$ it directs training in the wrong direction [2].

Whether such training expands ability or concentrates it is contested. Yue et al. [17] report that a verifiable reward raises single-sample accuracy while leaving coverage at large K at or below the base model, and ProRL [11] reports the opposite under a much longer schedule. Both measure coverage by sampling, so the number of distinct candidates a sampler happens to produce enters the measurement; structured search is a second way to raise that number [16]. This work removes the sampling term by scoring the complete candidate space and measures the contribution of structure against a baseline matched on diversity.

Compositional generalisation means combining mastered transformations into a sequential program. Separate command of A and B does not guarantee correct execution of A after B : order changes the result and an early error propagates. Lake and Baroni [9] show that sequence models generalise poorly to held-out combinations of familiar primitives, Zhao et al. [19] study whether skill composition can be learned from examples, and Dziri et al. [3] trace compositional failures to the structure of the task graph. This work uses the discover-and-distill scheme: exhaustive enumeration and an exact checking program determine the correct trajectories, and the model is fine-tuned on them with a low-rank adapter [7], receiving the full target answer including the operations and the verifiable intermediate states. Training a model on its own outputs can degrade it [13]; the loop here is protected by construction, since an exact checking program admits only correct trajectories.

§3. ENVIRONMENT AND DATA

The state is a polynomial of degree at most d over the finite field F_p , stored as the vector of its coefficients $[c_0, \dots, c_d]$ taken modulo p , with the length preserved including leading zeros. Five exact atomic operations are defined over this state: SH1 shifts the argument by one, SC2 scales it by two, REV reverses the coefficient vector, and AC1 and AX1 add one to the free coefficient and to the coefficient at x . All are computed deterministically, do not increase the size of the state and admit exhaustive enumeration of programs of small depth (Table 1).

The frozen protocol of the project fixes the rest of the environment for both series. Training uses the fields 5, 7, 13, 19, 23 and 29, the fields 11 and 17 are held back for transfer, and three *motifs* are held back from supervision, a motif being

an adjacent ordered pair of operations. The three are AX1 followed by SH1, AC1 followed by REV, and SC2 followed by AX1. At depth two the space contains $5^2 = 25$ programs, at depth three 125, and at depth four 625. Appendix A works through one program of each depth and gives a target that two different programs reach.

In PLAN mode the model receives the field, the start and target vectors, the list of admissible operations and the fixed program length, and must return a sequence of operations; in APPLY mode it is given an explicit sequence and must compute the final vector. APPLY was used to control the preservation of atomic skills, and the main compositional result was measured in PLAN mode.

The training set of the confirmatory series included 4000 composition tasks, and the final sets were generated separately, frozen before the series and not used when selecting hyper-parameters. Four families separate different kinds of transfer: A tests new input states under familiar compositional signatures and familiar fields, B holds local motifs under familiar fields, C uses familiar signatures in new fields, and D combines the two, with 1000 tasks each in A and B and 500 each in C and D. Appendix B gives their structure and the integrity checks.

§4. METHOD

4.1. Compared branches. Each independent run started from one frozen atomic adapter and split into two branches of equal budget. The main branch received full correct composition trajectories and 20 percent atomic replay; the control branch received an equal number of training tokens and steps on atomic tasks alone, so a difference in the result cannot be explained by a longer period of training. Both branches used low-rank adaptation with identical settings and identical base weights, and new adapters were created for each run. Figure 3 and Table 7 in Appendix B state the design and the settings.

4.2. Exact ranking and metrics. The evaluation did not use random generation and did not depend on random repetitions of one and the same program. For every task all distinct programs of the fixed depth were enumerated. The model score of a program is the sum of the conditional logarithms of the probabilities of its actions,

$$L(o_1, \dots, o_d \mid x) = \sum_{i=1}^d \log p_{\theta}(o_i \mid x, o_1, \dots, o_{i-1}). \quad (1)$$

Programs were sorted by decreasing score, ties were broken by the lexicographic order of the operation signatures, and the checking program marked all correct variants. The main measure Hit@32 is the share of tasks for which at least one

correct program enters the first 32 positions. Unlike $\text{random pass}@K$ [1] it does not depend on the repeated generation of identical strings. Depth two has only 25 candidate programs, so $\text{Hit}@32$ is structurally saturated there and a zero difference on it carries no information about the absence of an effect.

4.3. Statistical plan. The independent unit of confirmation was the training run: a thousand tasks inside one run are not a thousand independent models. For each of the six runs the difference in $\text{Hit}@32$ between the branches was computed on the same 1000 tasks of family A, and over the six differences the mean, the 95 percent t interval and the two-sided one-sample t test were computed. An exact sign-permutation test over all $2^6 = 64$ sign combinations and a two-level resampling over both runs and tasks served as robustness checks, and the Holm correction was applied to five secondary measures [4, 6]. The conditions for a positive result were fixed before the final set was opened; Appendix B gives them with the resampling detail.

§5. RESULTS

5.1. Main confirmatory result. The starting atomic adapter reached 99.8 percent in PLAN mode and 91.5 percent in APPLY mode, its weakest atomic operation being the shift SH1 at 57.5 percent.

On the main set A of depth three the control branch solved on average 45.32 percent of the tasks on $\text{Hit}@32$, and after fine-tuning on verified composition trajectories the mean was 72.72 percent. The absolute gain equals 27.40 percentage points, a factor of 1.60 in the share of successfully ranked tasks. The 95 percent t interval over the six independent runs equals [25.403, 29.397] percentage points and the two-sided p value of the one-sample t test equals 3.445×10^{-7} . All six runs gave a positive difference, the exact sign-permutation test gave $p = 0.03125$, and the two-level resampling gave the wider interval [23.117, 31.667]. Both intervals lie entirely above the preregistered threshold of no effect, and all preregistered conditions of a positive result were met. Figure 1 shows the individual runs.

The effect appeared in a continuous measure as well. In all six runs the normalised probability mass of the correct set grew, by 4.41 percentage points on average with a 95 percent t interval of [4.25, 4.57]. The model therefore did not merely move a single program across the boundary at position 32.

5.2. Boundaries of transfer. The secondary sets determine the boundaries of the effect, and Figure 6 in Appendix C collects them. On family C, where the composition families stayed familiar but new finite fields were used, the gain

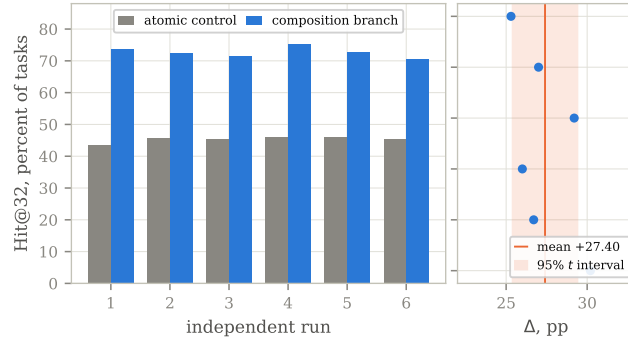


Figure 1. Left, Hit@32 of the two branches in each of the six independent runs, on the same 1000 tasks of family A at depth three. Right, the six per-run differences, the mean of 27.40 percentage points and the 95 percent t interval computed over them. The spread runs from 25.3 to 30.2 percentage points.

was 26.13 percentage points, positive in all six runs, which confirms transfer of the mastered compositional regularities to new arithmetic fields and shows that training did not reduce to memorising specific coefficients or a specific modulus. On family B with local motifs held back in advance the result fell by 16.02 points, and on family D, which combined held motifs and new fields, by 11.80. Fine-tuning therefore consolidated familiar families of composition trajectories and did not form a universal rule for motifs completely excluded from the training data. On the combined set of depth four a positive gain of 5.20 points was obtained. After the Holm correction the three depth-3 rows and the depth-4 row have p values of 8.487×10^{-5} , 1.879×10^{-7} , 1.024×10^{-5} and 2.976×10^{-5} .

5.3. Preservation of atomic skills. The positive compositional result was accompanied by partial forgetting. After the control atomic training the mean PLAN was 99.72 percent and the mean APPLY 90.28; after composition fine-tuning PLAN was 97.78 and APPLY 85.70. The main contribution came from SH1, which the control branch left at 51.5 percent and the composition branch at 29.08, while the remaining four operations stayed at 99 to 100 percent, and only two of the twelve trained branch-and-run pairs met the descriptive forgetting threshold

of two percentage points. In the final preregistration atomic forgetting was defined as descriptive and not blocking, and an earlier exploratory pilot on the configuration this series froze remains formally negative under its own older atomic-forgetting gate.

5.4. What the surrounding pipeline contributes. A second series, on the same base model and the same field division but with its own independently generated splits, three training seeds, 300-task evaluation sets for the depth and field grid and 250-task sets for the motif families, and its own shared atomic checkpoint as the reference that every gain below is measured against, measures the contribution of the search and the reward around the fine-tuning. Its numbers are not comparable with the intervals above, because it uses a different unit of replication and a crossed seed-by-task bootstrap. Figure 7 collects its contrasts, and Appendix E gives the tables.

Three of its measurements bound the reading of the first series. The advantage of structured exploration over independent sampling, reported by an earlier pilot at 16.0 percentage points and reproduced at 16.33, becomes -1.11 points with an interval of $[-4.22, +1.89]$ once the baseline is tempered to the diversity that structured search realises. Policy-gradient training [12] at the exploration optimum changes held-out coverage after 400 steps by $+0.22$ points, interval $[-3.11, +3.67]$, while raising per-candidate accuracy from 0.0151 to 0.0358 and lowering distinct programs per task from 3.80 to 1.81, a redistribution of probability inside the existing support. Direct supervision reproduces the confirmatory result at 48.22 and 59.11 points while an equal-budget atomic control, trained on an atomic pool that covers the fields 5 and 7 alone, moves the same measurement by $+0.11$ and $+1.67$ with both intervals containing zero; that study’s own registered retention rule does not count the gain as a success, because the adapters that produce it lose the APPLY mode entirely, and the motif axis reproduces at -13.47 and -15.07 points with a motif main effect of $+71.27$ against a field main effect of -1.00 . Adapters whose data contained the three withheld pairs reach Hit@32 of 0.935 and 0.933 on exactly those families, so the failure follows from the absence of the pair from supervision and not from a property of the tasks, and the contrast is sharp: the fields 11 and 17 are absent from every training file in the same sense and cost one point, while the withheld pairs cost 71. Appendix E gives the tables and the registered status of each study.

§6. DISCUSSION

The main result shows that full verified composition trajectories can be transferred into the parameters of a small language model. Under an unchanged evaluation procedure the correct programs moved upward in the complete ranked space, so the effect cannot be explained by a wider search, by the removal of repetitions or by more candidates during checking. The control branch received the same budget on fresh atomic tasks, so the difference of 27.40 percentage points reflects the information content of the composition trajectories and not an additional number of optimiser steps, and the equal-budget control of the second series measures the same thing on independently generated data. Family A contains new states and tasks under familiar signatures and familiar fields, so the result should be called a transfer of verified trajectories to new inputs, and not a proof of the mastery of an arbitrary previously unseen composition. The negative results on B and D show that local motifs excluded in advance were not derived automatically from the separate atomic skills.

The improvement was accompanied by a deterioration in part of the atomic skills, which agrees with the known problem of catastrophic forgetting under sequential fine-tuning [8], and 20 percent atomic replay did not ensure the full preservation of SH1. Two statements have to be separated: the positive compositional result is statistically confirmed in the preregistered main check, and the final model is at the same time not unconditionally better on all skills. Taken together with the degraded-verifier levels of Appendix E, the second series locates the constraint of the setting: a checking program that accepts every correct trajectory and rejects every incorrect one does not let policy-gradient training expand the set of reachable programs, and one corrupted at one label in two still permits direct supervision to stand 41 points above the untrained baseline. In this environment the binding constraint lies on the side of generation, and the unit in which the reachable set grows is the adjacent pair of operations that supervision demonstrates.

6.1. Limitations. The study uses one architecture and one model size, and transfer to other models requires separate verification. The main positive result concerns familiar families of compositional signatures on new inputs; transfer to held local motifs is not confirmed, and three motifs were withheld out of the 25

adjacent pairs. Atomic forgetting was substantial, especially for SH1. Six independent runs are sufficient for the criteria fixed in advance, and the exact sign-permutation test has a coarse resolution at that number. The environment is synthetic: its advantage is exact checking and complete enumeration. Appendix D gives the full list.

§7. CONCLUSION

A preregistered confirmatory experiment on the transfer of verified composition trajectories into the parameters of a small language model was carried out, using Qwen3-0.6B, a controlled environment of polynomial transformations over finite fields, an exact checking program and the complete ranking of all programs of a fixed depth. The main hypothesis is confirmed: on 1000 new tasks of family A the mean Hit@32 after composition fine-tuning was 72.72 percent against 45.32 in the control branch of equal budget, a gain of 27.40 percentage points with a 95 percent t interval of [25.403, 29.397] and $p = 3.445 \times 10^{-7}$, positive in all six runs, with transfer to new finite fields at 26.13 points. Transfer to local motifs held back in advance is not confirmed, which bounds the domain of the conclusion and shows that the mastery of familiar composition families is not identical to universal compositional generalisation. The second series adds that structured exploration carries no advantage at matched diversity, that four hundred steps with an exact reward sharpen the policy by 2.37 times per candidate and narrow it by 2.10 while leaving coverage unchanged within three points, and that an equal-budget atomic control moves the composition measure by 0.11 and 1.67 points. Exact verified composition trajectories can therefore be consolidated in the parameters of a small language model and can substantially raise the rank of correct programs on new tasks under an unchanged evaluation procedure; what bounds the result in this environment is the set of programs the model can produce at all.

§8. CONTRIBUTION

Artem Churilkin proposed and implemented the measurement the paper is built on: the exhaustive enumeration of every program of a given depth, the selection of the correct ones by the exact checking program, and the ranking metric built on them, which asks whether fine-tuning placed those programs higher and not whether a sampler happened to find them. He ran the fine-tuning on full correct trajectories and the control training on known-correct solutions, which separates whether the model can represent these compositions from whether a given method finds them; designed the decomposition that varies program depth

and finite field on separate axes instead of both at once; and ran the final comparison of methods at an equal candidate budget across several seeds. He wrote the manuscript.

Co-author Two REPLACE THIS WITH YOUR OWN CONTRIBUTION.

Co-author Three REPLACE THIS WITH YOUR OWN CONTRIBUTION.

All authors agreed on the claims the paper makes, on the boundary between what the experiments establish and what they do not, and on the retention of the negative results in full.

ACKNOWLEDGMENTS

We thank our mentor **Serguei Barannikov** for the problem statement and for the protocol behind this study, carried out within SMILES 2026.

APPENDIX §A. WORKED EXAMPLES OF THE ENVIRONMENT

Table 2 shows all five transformations on one starting polynomial over F_{11} , which makes it visible that they change different parts of the state. Figure 2 traces one shortest program of depth three and gives one target reached by two different programs of depth four. Table 3 collects further correct compositions across fields, degrees and depths. Every row was re-executed with the deterministic interpreter of the experiments, and every uniqueness claim was re-checked by exhaustive search over all programs of that depth and shorter.

Code	Transformation	Effect
SH1	$P(x) \mapsto P(x + 1)$	shift of the argument by one
SC2	$P(x) \mapsto P(2x)$	scaling of the argument
REV	$[c_0, \dots, c_d] \mapsto [c_d, \dots, c_0]$	reversal of the vector
AC1	$c_0 \mapsto c_0 + 1$	change of the free coefficient
AX1	$c_1 \mapsto c_1 + 1$	change of the coefficient at x

Table 1. The five atomic transformations, all taken modulo p . Each one is computed deterministically and none of them changes the length of the state vector, so programs of small depth can be enumerated in full. AC1 and AX1 look similar and touch different coordinates, so after REV or SH1 they lead to different final states.

Code	Exact action on $P(x) = 2 + 5x + 3x^2 + 4x^3$	Result
SH1	shift of the argument, $P(x) \rightarrow P(x + 1): 2 + 5(x + 1) + 3(x + 1)^2 + 4(x + 1)^3 = 14 + 23x + 15x^2 + 4x^3$	[3, 1, 4, 4]
SC2	scale of the argument, $P(x) \rightarrow P(2x)$: the coefficient at x^i is multiplied by 2^i , giving [2, 10, 12, 32]	[2, 10, 1, 10]
REV	reversal of the coefficient vector. This is neither a reversal of digits nor a substitution of $-x$ for x : $[c_0, c_1, c_2, c_3] \rightarrow [c_3, c_2, c_1, c_0]$	[4, 3, 5, 2]
AC1	one is added to the free coefficient only: $[2 + 1, 5, 3, 4]$	[3, 5, 3, 4]
AX1	one is added to the coefficient at x only: $[2, 5 + 1, 3, 4]$	[2, 6, 3, 4]

Table 2. All five transformations on one starting polynomial over F_{11} , with the state [2, 5, 3, 4]. Coefficients are reduced modulo 11 after every operation. AC1 and AX1 look similar and change different coordinates, so after REV or SH1 they lead to different final states. Every row was re-executed with the same deterministic interpreter that the experiments use.

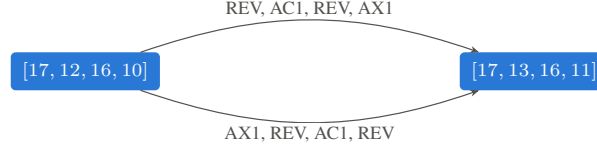
(a) one shortest program over F_{11} (b) two shortest programs for one target over F_{19} 

Figure 2. Panel (a) gives every intermediate state of one shortest program of depth three over F_{11} . Order matters: exchanging the first two operations sends the start to $[5, 6, 8, 0]$ instead of the target. Panel (b) gives one target over F_{19} that two different programs of depth four reach. Because several programs may reach one target, Hit@32 uses the best rank among all correct programs.

Depth	p	Start	Target	A correct program	Type
2	7	[1, 2, 3]	[3, 2, 2]	AC1, REV	unique shortest
3	11	[0, 2, 2, 5]	[5, 4, 1, 0]	AX1, REV, SC2	unique shortest
3	7	[4, 4, 5, 6]	[0, 2, 4, 5]	SH1, REV, AC1	unique shortest
4	11	[10, 10, 2]	[0, 10, 7]	SC2, REV, SC2, SH1	unique shortest
4	23	[7, 8, 18, 5]	[16, 20, 1, 10]	REV, AX1, SH1, SC2	one solution
4	19	[17, 12, 16, 10]	[17, 13, 16, 11]	the two of Fig. 2	two solutions

Table 3. Correct compositions across different fields, degrees and depths. Every row was re-executed with the deterministic interpreter of the experiments, and every uniqueness claim was re-checked by exhaustive search over all programs of that depth and shorter. Order matters throughout: on the second row the sequence REV, AX1, SC2 gives $[5, 6, 8, 0]$ instead of the target, and on the fourth row exchanging the first two operations gives $[4, 8, 6]$.

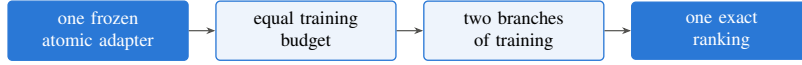


Figure 3. The logic of the confirmatory experiment. Every independent run starts from one and the same frozen atomic adapter, the two branches receive an identical number of optimiser steps, epochs, training tokens and an identical effective batch size, and both are then evaluated by one and the same exhaustive ranking. Only the type of training material differs between the branches.

Set	Signatures	Fields	Tasks	Role
A	familiar	familiar	1000	main test
B	held motifs	familiar	1000	transfer to new motifs
C	familiar	new	500	transfer between fields
D	held motifs	new	500	double transfer

Table 4. Structure of the final sets of the confirmatory series. All specific input states and tasks were new, the sets were frozen before the confirmatory series and were not used to select any hyper-parameter, and the leak audit found no intersection with the atomic, training and development sets. Check sets of 125 tasks at depths 2 and 4 were built for each family in addition.

Condition	Threshold
mean gain in Hit@32	at least 5 percentage points
lower bound of the 95 percent t interval	strictly above zero
two-sided p value	below 0.05
number of positive runs	at least 5 of 6
data integrity	full check, equal budget, no leaks

Table 5. Conditions for a positive result, fixed before the final set was opened, together with the main endpoint, the set A, the depth and the number of independent runs. All five were met.

APPENDIX §B. DESIGN, PREREGISTRATION AND STATISTICAL DETAIL

For a run s the difference of the main measure was defined as $\Delta_s = \text{Hit@32}_s(\text{composition}) - \text{Hit@32}_s(\text{control})$, and over the six differences

$$\bar{\Delta} = \frac{1}{6} \sum_s \Delta_s = 0.274, \quad \bar{\Delta} \pm t_{0.975,5} \frac{s\Delta}{\sqrt{6}} = [0.25403, 0.29397]. \quad (2)$$

Branch	Training data	Purpose
atomic control	fresh atomic tasks	control for the effect of further training and of the number of tokens
composition branch	full correct composition trajectories and 20 percent atomic replay	consolidation of verified operation sequences

Table 6. The two branches of the confirmatory series. Each independent run started from one frozen atomic adapter and split into these two. The correct composition trajectories were found by exhaustive search and confirmed by the deterministic verifier. The main branch received all the tokens of the program and the intermediate states, and the control branch received an equal number of training tokens and steps on atomic tasks alone, so a difference in the result cannot be explained by a longer period of training.

Parameter	Value
LoRA rank	32
scaling factor	64
dropout probability	0.05
learning rate	1×10^{-4}
epochs	2
effective batch size	64
warm-up fraction	3 percent
gradient-norm clip	1.0
arithmetic format	bfloat16
atomic replay in the main branch	20 percent

Table 7. Fine-tuning parameters of the confirmatory series, from its frozen protocol. Both branches used low-rank adaptation with these settings and the base weights stayed identical, and new adapters were created for every independent run so that stability across runs could be assessed.

The two-level resampling redrew both runs and tasks, used the same resampled sequence of task indices in all selected runs so that the paired structure was preserved, and ran 20000 repetitions, giving the percentile interval [0.23117, 0.31667].

Quantity	Value
mean gain	+27.40 pp
95 percent t interval	[25.403, 29.397] pp
t , degrees of freedom	35.275, df = 5
two-sided p from the t test	3.445×10^{-7}
positive runs	6 of 6
exact sign-permutation test	$p = 0.03125$
two-level bootstrap 95 percent interval	[23.117, 31.667] pp

Table 8. Statistical confirmation of the main effect on set A at depth 3. The unit of replication is the training run, so the interval and the p value are computed over the six differences between branches. The mean gain, the t interval and the t statistic in this table were recomputed here from the per-run values of Fig. 1 and agree with the values reported by that series to three decimals. With six runs the smallest attainable two-sided p of the exact sign-permutation test is 0.03125, which is the value obtained.

Run	Both	Control only	Composition only	Neither	Δ , pp
1	305	130	432	133	+30.2
2	316	141	408	135	+26.7
3	305	149	409	137	+26.0
4	343	117	409	131	+29.2
5	321	138	408	133	+27.0
6	325	129	382	164	+25.3

Table 9. Number of the 1000 tasks of set A solved by both branches, by the control branch only, by the composition branch only, and by neither. The difference in Hit@32 equals the count solved only by the main branch minus the count solved only by the control, divided by 1000. Each row sums to 1000 and reproduces the per-run values of Fig. 1, which was checked here and not assumed.

APPENDIX §C. ATOMIC SKILLS AND DATA INTEGRITY

The full re-check covered 2080 shard files of rankings, verifying for every task the completeness of the enumeration of 25, 125 or 625 programs, the correctness of the labels and the sort order. The leak audit gave zero intersections,

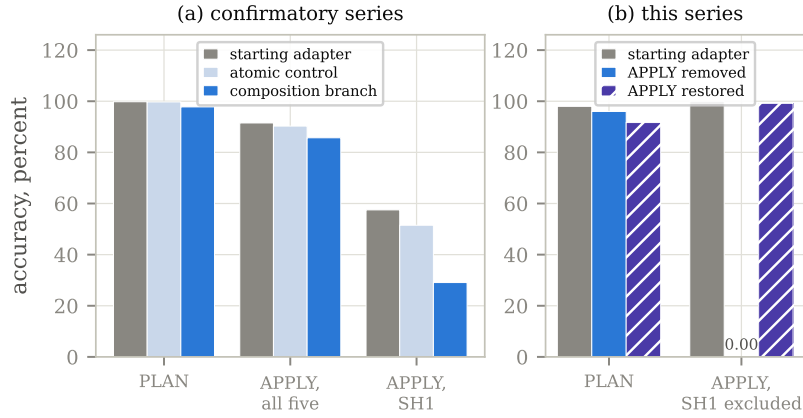


Figure 4. Retention of the atomic modes in the two series. The panels are not directly comparable: panel (a) averages APPLY over all five operations while panel (b) uses a split of the second series from which SH1 is excluded, and the two also differ in the training mixture.

and the training budgets inside each pair of branches were equal in optimiser steps, epochs, effective batch size and loss-bearing target tokens. Acceleration on three RTX 5080 cards was applied only to a part of the secondary rankings, and those data were reprocessed by the frozen evaluation code. The compact audit archive publishes aggregated results and the checksums of the 2080 shards, and not the individual rows of the main family-A set for the composition branch, so no rank of an individual task of that set is attributed anywhere in this paper.

APPENDIX §D. FULL LIST OF LIMITATIONS

- (1) The study was carried out on one architecture and one model size. Transfer of the result to other models requires separate verification.
- (2) The main positive result concerns familiar families of compositional signatures on new inputs. Transfer to held local motifs is not confirmed.
- (3) The composition environment is synthetic. Its advantage is exact checking and the complete enumeration of programs; real mathematical reasoning has a more complex action space.

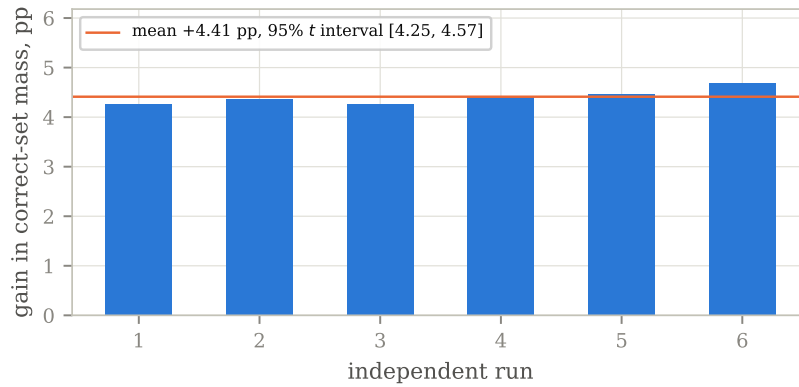


Figure 5. Gain in the normalised probability mass of the correct set in each of the six independent runs, on the 1000 tasks of family A. The mean and the interval were recomputed from the six per-run values.

Check	Result
six independent runs completed	yes
equality of the training budgets	confirmed
identical tasks for branches and runs	confirmed
number of leaks found	0
re-checked ranking shards	2080
completeness of the depth 2, 3 and 4 spaces	25, 125 and 625 programs
re-check of labels and of the sort order	passed
preregistration created before the final data	yes
pilot adapters used in the confirmation	no
random generation at the main evaluation	not used
correction for multiple secondary tests	Holm
negative secondary results published	yes

Table 10. Technical audit and reproducibility control of the confirmatory series. The full re-check covered 2080 shard files of raw rankings, and for every task it re-verified the completeness of the enumeration, the labels of the verifier and the sort order.

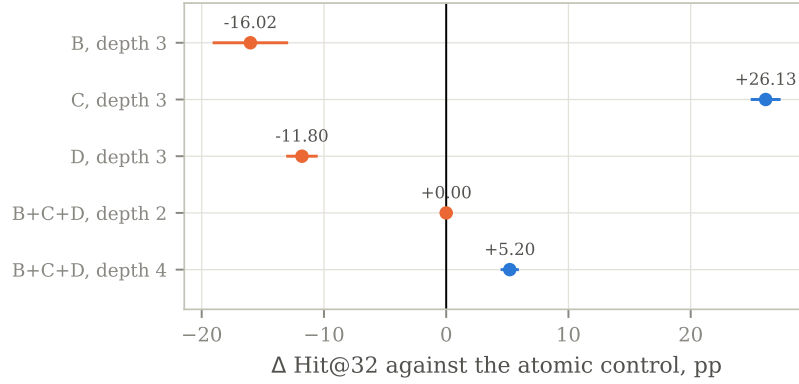


Figure 6. Secondary measures of transfer with their 95 percent intervals, as differences against the atomic control. Family B holds local motifs under familiar fields, C uses familiar signatures in new fields, and D combines the two. The depth-2 row is structurally saturated, so its zero carries no information. After the Holm correction the three depth-3 rows and the depth-4 row have p values of 8.487×10^{-5} , 1.879×10^{-7} , 1.024×10^{-5} and 2.976×10^{-5} .

- (4) Atomic forgetting was substantial, especially for SH1. The method requires an improved mechanism for preserving the base skills.
- (5) The number of independent runs of the confirmatory series equals six, which is sufficient for the criteria fixed in advance, and the exact sign-permutation test has a coarse resolution at that number of runs.
- (6) The second series uses three seeds and 300 tasks per split, which resolves a contrast to roughly three percentage points. An interval containing zero means that smaller effects would not have been detected.
- (7) Its self-training result covers one algorithm at one budget and one schedule. It states that policy-gradient training under these conditions does not expand coverage, and not that reinforcement learning cannot.
- (8) Three motifs were withheld out of the 25 adjacent pairs, the same three in both series, and the size of the motif effect requires separate verification for the rest.

- (9) The verifier levels use one training seed each, and the boundary conditions of Appendix E reuse evaluation sets that had already been opened. They are reported as post-hoc controls.
- (10) The positive result should not be called the creation of a universal new ability. It proves the consolidation and transfer of verified trajectories within a clearly defined domain.

APPENDIX §E. THE SECOND SERIES

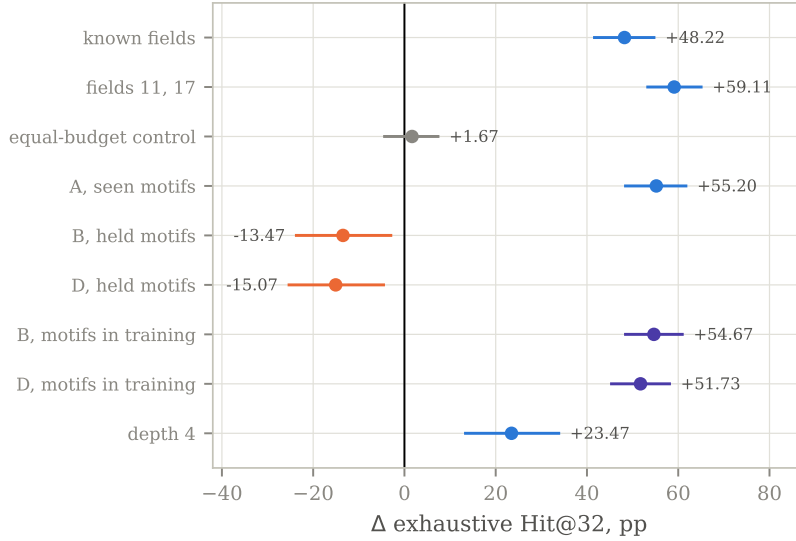


Figure 7. Contrasts of the second series in exhaustive Hit@32 with their 95 percent crossed seed-by-task intervals. Rows in the darker colour are measured against the shared starting checkpoint, the grey row is an equal-budget atomic control, and the two rows in green are the same held-motif families ranked by adapters whose supervision contained the withheld pairs.

The second series holds the same base model, the same operation set and the same field division. It uses three training seeds, 300-task evaluation splits for the depth and field grid and 250-task splits for the four motif families, and a 10000-fold crossed seed-by-task bootstrap. Its supervised runs take 1875 optimiser steps

at a batch of 8 and a learning rate of 2×10^{-4} ; its policy-gradient runs take 400 steps with a group size of 8, a learning rate of 1×10^{-5} and a Kullback-Leibler coefficient of 0.01. Its numbers are therefore not comparable with the six-run intervals of the main text.

The two series also score programs differently: the confirmatory series reads the log-probability of an operation off a softmax over the full vocabulary with five added operation tokens, while the second series takes a softmax over the five admissible continuations alone. Both rank by the same summed quantity, and the normaliser that the first omits can reorder programs relative to the second, which bounds how literally "reproduces" should be read across the two. An audit of the second series' ten splits by exact key recorded overlaps in seven pairs, one of them a training-and-evaluation leak of a single item; recomputed on never-trained items only the gate metrics are 1.00 on 72 items, 0.9775 on 89 and 0.8488 on 86, so the overlaps did not inflate them.

Studies E1 and E2 were preregistered in the ordinary sense. E3 was preregistered with sampled pass@32 as its primary endpoint, and an amendment moved the primary weight to exhaustive Hit@32; the sampled contrasts are +63.33 and +62.56 percentage points against the exhaustive +48.22 and +59.11. E4, the motif axis, was added by that same amendment, and its withheld set of three operation pairs was taken from the confirmatory series after that series' own result was open, so E4 tests a hypothesis chosen with knowledge of the effect on data generated independently of it. The amendment file also carries a modification time about an hour after the last of E3's primary ranking cells finished, so the order in which the amendment was written and E3's data were read cannot be established from the artifacts. The boundary conditions below reuse sets that E3 and E4 had already opened and are post-hoc throughout.

	Intervention	Evaluation	Status
E1	search method at matched candidate diversity	300 tasks, 3 seeds	confirmatory
E2	400 steps of GRPO with an exploring sampler	300 tasks, 3 seeds	confirmatory
E3	supervision on correct depth-2 and depth-3 programs	4×300 , 3 seeds	confirmatory
E4	removal of three operation pairs from supervision	4×250 , 3 seeds	confirmatory
E5	equal-budget control, motif-present control, APPLY mixture, depth 4, verifier noise	reused sets, 125 new tasks	post-hoc

Table 11. The five studies and their status, stated before any result is shown. E1 to E4 were preregistered, with the primary endpoint, the decision rule and the frozen set fixed in advance. E4 was added by an amendment whose position in time relative to the opening of E3’s data cannot be established from the artifacts, so it is a disclosed extension and not an independently preregistered test. E5 collects boundary conditions on sets that E3 and E4 had already opened, together with one newly generated depth-4 split, and is therefore reported as post-hoc. The verifier levels of E5 use one training seed each and the rest of E5 uses three. The claims of E1 to E4 stand without E5.

Search arm	pass@32	Distinct
iid, $T = 0.7$, the original control	0.060	3.11
iid, $T = 1.6$	0.190	8.04
iid, $T = 2.0$	0.250	9.88
iid, $T = 3.0$	0.390	14.57
iid, $T = 5.0$	0.460	19.17
iid, $T = 8.0$	0.500	21.92
iid, $T = 100.0$, near uniform	0.420	23.74
temperature mixture	0.090	4.25
prefix-balanced, $T = 0.7$	0.290	9.63

Table 12. Selection of the search arms on the development split at one seed: 100 PLAN tasks over the fields 5 and 7, of which 46 are of depth two and 54 of depth three, so the last column is an average over both depths. The last column counts distinct programs produced per task inside a budget of 32 draws. Temperature here divides a summed sequence log-probability, so these values are not comparable with ordinary decoding temperatures. The arm at $T = 2.0$ matches the realised diversity of prefix-balanced search and fixes T^* , and the interior optimum at $T = 8.0$ fixes the training sampler of E2. No confirmatory set was opened for this selection.

Contrast	Δ , pp	95% CI	Seeds
E1, structure against diversity			
Q1, prefix-balanced — iid at T^*	−1.11	[−4.22, +1.89]	0/3
Q2, GRPO effect, iid branch	−0.11	[−0.89, +0.67]	1/3
Q2, GRPO effect, prefix branch	−1.89	[−3.78, +0.00]	0/3
Q3, the earlier diagonal, new set	+9.00	[+6.89, +11.11]	3/3
Q3, the earlier diagonal, earlier set	+16.33	[+13.17, +19.50]	3/3
Q4, iid at $T = 3.0$ — prefix-balanced	+9.78	[+6.44, +13.00]	3/3
E2, self-training			
Q5, after GRPO — its starting point	+0.22	[−3.11, +3.67]	2/3
Q6, exploring — cold training sampler	+0.89	[−2.45, +4.22]	2/3
Q7, uniform random — prefix-balanced	+11.22	[+7.33, +15.11]	3/3
Q8, best iid — prefix-balanced	+12.67	[+8.89, +16.56]	3/3

Table 13. Preregistered contrasts in sampled pass@32, three training seeds. The two frozen exploration sets carry 300 depth-3 PLAN tasks each over fields 11 and 17; the row marked as the earlier set is a third, earlier frozen set of 200 tasks, re-analysed and not re-collected. Intervals come from a 10,000-fold hierarchical paired bootstrap over tasks and seeds. The last column counts the seeds whose own difference carries the sign of the pooled estimate. Q1 and Q5 are the two primary endpoints, and both fall in the preregistered branch where the interval contains zero.

Adapter	Accuracy	Distinct	pass@32
before GRPO	0.0151	3.80	0.044
after GRPO, $T_{\text{train}} = 0.7$	0.0139	2.49	0.038
after GRPO, $T_{\text{train}} = 8.0$	0.0358	1.81	0.061

Table 14. Decomposition of the null in Q5, measured at the search arm $T = 0.7$ where the preferences of the policy dominate the candidate set. Accuracy is the fraction of drawn candidates the verifier accepts, and the third column counts distinct programs per task inside a budget of 32 draws. The exploring policy is 2.37 times more accurate per candidate and 2.10 times narrower.

Contrast in exhaustive Hit@32	Δ , pp	95% CI
E3, against the shared starting checkpoint		
supervision, known fields	+48.22	[+41.33, +55.00]
supervision, fields 11 and 17	+59.11	[+53.00, +65.33]
known — held-out fields, after supervision	−2.22	[−6.00, +1.56]
depth 2 — depth 3 at Hit@8, fields fixed	+42.89	[+38.00, +47.67]
E5, against an equal-budget atomic control		
atomic control, known fields	+0.11	[−6.44, +6.44]
atomic control, fields 11 and 17	+1.67	[−4.67, +7.67]
supervision — atomic control, known fields	+48.11	[+38.67, +57.11]
supervision — atomic control, fields 11, 17	+57.44	[+49.67, +65.33]

Table 15. Direct supervision on correct depth-3 programs, 300 tasks per split and three training seeds, with all 125 candidate programs ranked exactly. The first two rows of each block compare a trained adapter with the reference named in the block heading. The atomic control received the same 1875 optimiser steps, the same learning rate and the same number of training examples on atomic data alone, so the lower block separates the content of the supervision from the length of the optimisation.

Family	Fields, motifs	Hit@32	Δ , pp	95% CI
supervision with the three operation pairs withheld				
A	known, seen	0.940	+55.20	[+48.13, +62.00]
B	known, held	0.253	−13.47	[−24.00, −2.67]
C	11 and 17, seen	0.936	+58.80	[+51.47, +65.73]
D	11 and 17, held	0.265	−15.07	[−25.60, −4.27]
supervision that contained those pairs				
B	known, held	0.935	+54.67	[+48.13, +61.20]
D	11 and 17, held	0.933	+51.73	[+45.07, +58.40]

Table 16. The motif experiment, 250 depth-3 tasks per family and three training seeds. A task enters a held family only when every minimal-depth correct program for it contains one of the three withheld pairs, so an unheld route cannot solve it. Differences are taken against the shared starting checkpoint, which reaches Hit@32 of 0.388, 0.388, 0.348 and 0.416 on families A to D. The upper block is E4 and the lower block is the control of E5 on the same two evaluation families. The motif main effect over the upper block is +71.27 pp, 95% CI [+58.53, +83.60], and the field main effect is −1.00 pp, 95% CI [−8.27, +6.33].

Training mixture	APPLY	Parse	PLAN	Vectors
no composition supervision	1.000	1.000	0.980	80/80
APPLY absent from the stream	0.000	0.000	0.910–0.980	0/480
APPLY at 17 percent of the stream	0.975–1.000	1.000	0.850–0.970	240/240

Table 17. The atomic modes on the frozen validation splits, 80 APPLY tasks and 100 PLAN tasks, at greedy decoding. The APPLY and PLAN columns give pass@1 and the Parse column gives the parse rate of the APPLY answers. The last column counts answers to an APPLY prompt that contain a coefficient vector at all, pooled over the six adapters of the middle row and the three of the last row.

Adapter	Hit@1	Hit@8	Hit@32	Best rank
no supervision	0.000	0.016	0.056	237.6
supervision, seed 0	0.040	0.128	0.336	92.8
supervision, seed 1	0.024	0.104	0.336	88.9
supervision, seed 2	0.016	0.072	0.200	143.2

Table 18. Exhaustive ranking of all 625 programs of depth 4 on 125 tasks over known fields, where supervision had covered depths 2 and 3 only. The last column is the mean rank of the best correct program. The pooled difference in Hit@32 is +23.47 pp, 95% CI [+13.07, +34.13], and the three seeds are listed separately because they do not agree closely.

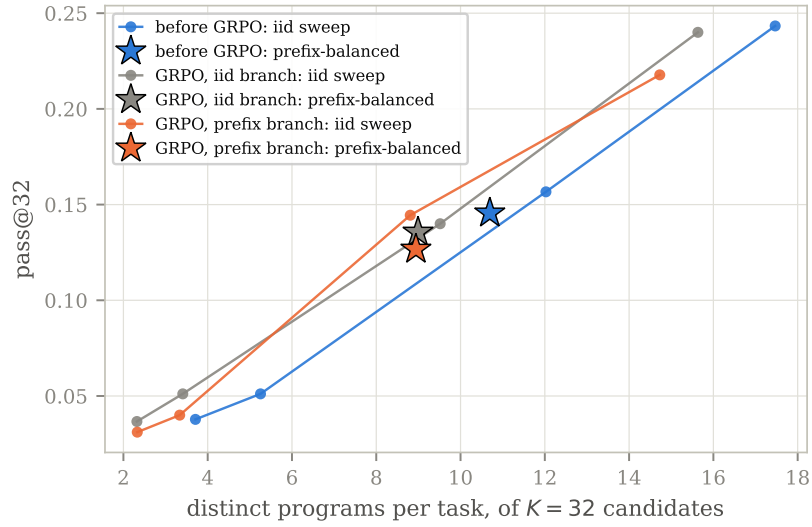


Figure 8. Coverage of the confirmatory set of the second series against realised candidate diversity, 300 depth-3 PLAN tasks over fields 11 and 17 at $K = 32$, averaged over three seeds. The stars, which mark prefix-balanced search, lie on the curve traced by the sweep over independent sampling temperatures.

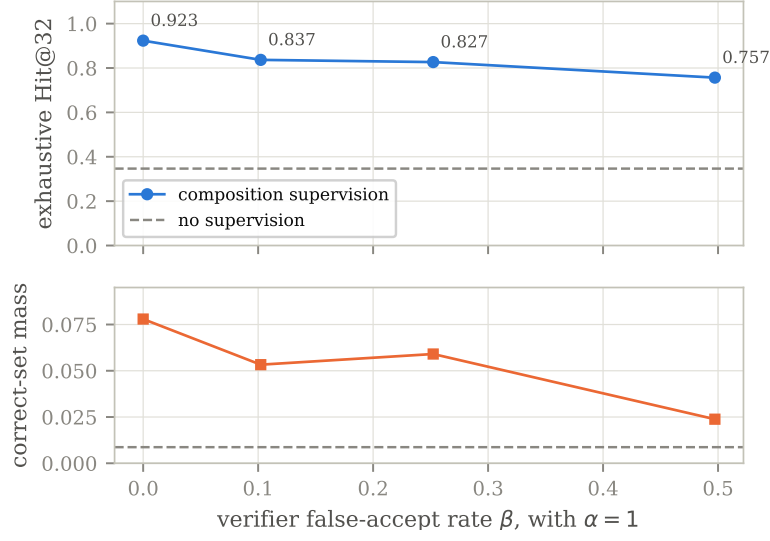


Figure 9. Composition supervision under a checking program that accepts every correct trajectory and a fraction β of incorrect ones, on the 300 tasks of depth three over fields 11 and 17. One training seed per level, so this figure is exploratory and no interval over seeds is available.

REFERENCES

REFERENCES

1. Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, et al., *Evaluating large language models trained on code*, arXiv preprint arXiv:2107.03374, 2021.
2. Thomas M. Cover and Joy A. Thomas, *Elements of information theory*, 2nd ed., Wiley, 2006.
3. Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, et al., *Faith and fate: Limits of transformers on compositionality*, Advances in Neural Information Processing Systems (NeurIPS), 2023.
4. Bradley Efron and Robert J. Tibshirani, *An introduction to the bootstrap*, Chapman and Hall, 1993.
5. Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, et al., *Reinforced self-training (ReST) for language modeling*, arXiv preprint arXiv:2308.08998, 2023.
6. Sture Holm, *A simple sequentially rejective multiple test procedure*, Scandinavian Journal of Statistics **6** (1979), no. 2, 65–70.
7. Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen, *LoRA: Low-rank adaptation of large language models*, Proceedings of the International Conference on Learning Representations (ICLR), 2022.
8. James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, et al., *Overcoming catastrophic forgetting in neural networks*, Proceedings of the National Academy of Sciences **114** (2017), no. 13, 3521–3526.
9. Brenden M. Lake and Marco Baroni, *Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks*, Proceedings of the International Conference on Machine Learning (ICML), 2018, pp. 2873–2882.
10. Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe, *Let’s verify step by step*, Proceedings of the International Conference on Learning Representations (ICLR), 2024.
11. Mingjie Liu, Shizhe Diao, Ximing Lu, Jian Hu, Xin Dong, Yejin Choi, Jan Kautz, and Yi Dong, *ProRL: Prolonged reinforcement learning expands reasoning boundaries in large language models*, Advances in Neural Information Processing Systems (NeurIPS), 2025.
12. Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo, *DeepSeekMath: Pushing the limits of mathematical reasoning in open language models*, arXiv preprint arXiv:2402.03300, 2024.
13. Ilya Shumailov, Zakhar Shumaylov, Yiren Zhao, Nicolas Papernot, Ross Anderson, and Yarin Gal, *AI models collapse when trained on recursively generated data*, Nature **631** (2024), 755–759.
14. Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou, *Self-consistency improves chain of thought reasoning in language models*, Proceedings of the International Conference on Learning Representations (ICLR), 2023.
15. An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, et al., *Qwen3 technical report*, arXiv preprint arXiv:2505.09388, 2025.
16. Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan, *Tree of thoughts: Deliberate problem solving with large language models*, Advances in Neural Information Processing Systems (NeurIPS), 2023.

-
17. Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang, *Does reinforcement learning really incentivize reasoning capacity in large language models beyond the base model?*, Advances in Neural Information Processing Systems (NeurIPS), 2025.
 18. Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman, *STaR: Bootstrapping reasoning with reasoning*, Advances in Neural Information Processing Systems (NeurIPS), 2022.
 19. Haoyu Zhao, Simran Kaur, Dingli Yu, Anirudh Goyal, and Sanjeev Arora, *Can models learn skill composition from examples?*, Advances in Neural Information Processing Systems (NeurIPS), 2024.

(Artem Churilkin) ¹Affiliation

E-mail: artem@superpositions.tech